

# Network Intrusion Detection System using Machine Learning

Introduction to Open Source Software · Sejong University · June 2026

Instructor: Junaid Rashid

DATASET

**NSL-KDD**

LANGUAGE

**Python 3.13**

BEST ACCURACY

**77.07%**

## 1 Project Purpose & Description

A **Network Intrusion Detection System (NIDS)** monitors network traffic and identifies malicious activity automatically. With cyberattacks growing in frequency and sophistication, machine-learning-based detection has become a key tool in modern cybersecurity.

This project trains three classifiers on the **NSL-KDD benchmark dataset** to classify each network connection record as either **Normal** or an **Attack**. It produces visual reports and provides an interactive command-line interface for live predictions.

**Why is this useful?** Network administrators can use such a system to automatically flag suspicious connections, reducing manual log inspection and accelerating incident response time.



### Auto Dataset Download

NSL-KDD fetched automatically on first run



### 3 ML Models Trained

Logistic Regression, Decision Tree, Random Forest



### 6 Visualizations

Distribution, heatmap, confusion matrix & more



### Model Persistence

Best model saved as best\_model.pkl via joblib



### Interactive CLI

Enter traffic values → get instant prediction



### MIT Licensed

Fully open source, free to use and modify

## 2 Tools & Libraries Used

### numpy

Numerical arrays

### pandas

Data loading & manipulation

### scikit-learn

ML models & preprocessing

### matplotlib

Chart generation

### seaborn

Statistical heatmaps

### joblib

Model serialization

All libraries are free, open source, and installable in one command: `pip install -r requirements.txt`

## 3 Project Workflow

1

### Load Dataset

NSL-KDD TXT files auto-downloaded (125,973 training / 22,544 test records, 41 features each)

2

### Preprocess

Multi-class labels → binary (Normal=0 / Attack=1). Categorical columns label-encoded. All features scaled with StandardScaler.

3

### Train Models

Three classifiers trained: Logistic Regression (max\_iter=1000), Decision Tree (max\_depth=10), Random Forest (100 trees, n\_jobs=-1).

4

### Evaluate & Compare

Accuracy, precision, recall, F1-score computed on KDDTest+. Best model selected automatically by highest accuracy.

5

### Save & Visualize

Best model saved to models/. Six charts saved to outputs/ as PNG files.

6

### Interactive Prediction CLI

User enters network feature values → model predicts Normal or Attack with a confidence score.

### MAIN.PY — ENTRY POINT

Parses command-line arguments (`--train`, `--predict`, `--all`) and calls the appropriate pipeline functions. Displays an ASCII project banner on startup.

### TRAIN.PY — TRAINING PIPELINE

Handles everything from data download to model saving. Key functions: `download_dataset()`, `preprocess()`, `train_models()`, `evaluate_models()`, `save_best_model()`, and six `plot_*` functions.

### PREDICT.PY — PREDICTION CLI

Loads the saved model from disk. Offers a menu: run predefined sample inputs, manually enter feature values, or exit. Returns the prediction label plus a confidence percentage.

```
# Run full training pipeline (downloads dataset, trains 3 models, saves 6 charts)
py main.py --train

# Launch the interactive prediction CLI
py main.py --predict

# Train and then immediately open the prediction CLI
py main.py --all
```

## 5 Test Cases & Results

### Test Case 1 — Normal HTTP Traffic

✓ NORMAL

Protocol: **TCP**

Service: **HTTP**

src\_bytes: **215**

dst\_bytes: **45,076**

logged\_in: **1 (Yes)**

error\_rate: **0.00**

Confidence: **~94%**

### Test Case 2 — DoS Neptune Attack

🚨 ATTACK Detected

Protocol: **TCP**

Service: **HTTP**

src\_bytes: **0**

dst\_bytes: **0**

error\_rate: **1.00 (SYN flood)**

count: **511 connections**

Confidence: **~99%**

### FULL TEST SET RESULTS (KDDTEST+, 22,544 RECORDS)

| MODEL                       | ACCURACY      | NORMAL PRECISION | ATTACK RECALL | F1 SCORE    |
|-----------------------------|---------------|------------------|---------------|-------------|
| Logistic Regression         | 75.39%        | 0.65             | 0.62          | 0.75        |
| Decision Tree               | 76.34%        | 0.65             | 0.61          | 0.76        |
| <b>Random Forest ✓ Best</b> | <b>77.07%</b> | <b>0.66</b>      | <b>0.62</b>   | <b>0.77</b> |

**Why ~77% instead of ~99%?** The NSL-KDD KDDTest+ file intentionally contains novel attack categories not present in the training set — simulating real-world conditions where new attack types emerge. On a standard 80/20 train-test split of the training data, accuracy exceeds 99%.

### MODEL ACCURACY COMPARISON



## 6 Why README.md and requirements.txt Matter

---

### README.MD

The README is the **front page** of every open source project. It answers the three most important questions a new user has: *What does this do? How do I install it? How do I use it?* On platforms like GitHub, the README renders automatically — it is often the first and only thing a visitor reads before deciding to use or contribute to a project.

### REQUIREMENTS.TXT

This file lists every Python library the project depends on with version constraints. Running `pip install -r requirements.txt` recreates the exact environment on any machine in seconds. This guarantees **reproducibility** — a core principle of open source software — and eliminates the common *"works on my machine"* problem.